

SELENIUM AUTOMATION ASSIGNMENT

Assignment 2 – Synchronization & Explicit Waits

Execution Report • Selenium with Python • Dynamic Loading

Item	Details
Assignment	2 – Synchronization & Explicit Waits
Application Under Test	The Internet – Dynamic Loading
Automation Tool	Selenium WebDriver
Programming Language	Python
Browser	Google Chrome
Synchronization	WebDriverWait + expected_conditions
Restriction	No time.sleep()
Validation	Extracted text equals 'Hello World!'
Execution Status	PASSED

This report documents the implementation, synchronization strategy, source code, execution evidence, and validation result for the dynamic-content automation task.

1. Objective

The objective of this assignment is to automate interaction with a page containing dynamic content and demonstrate proper synchronization using an explicit wait.

The test must click the Start button, wait for the dynamically generated text to become visible, extract the text, and validate the result. The assignment specifically prohibits the use of `time.sleep()`, so the synchronization must be implemented using `WebDriverWait` and Selenium `expected_conditions`.

2. Assignment Requirements

No.	Requirement	Implementation
1	Open a dynamic-content page	The Internet – Dynamic Loading
2	Start the dynamic operation	Click the Start button
3	Avoid fixed delays	No <code>time.sleep()</code> used
4	Use explicit synchronization	<code>WebDriverWait(driver, 10)</code>
5	Use an expected condition	<code>visibility_of_element_located()</code>
6	Extract the dynamic text	<code>element.text</code>
7	Validate the result	Assert text equals 'Hello World!'

3. Test Environment

Component	Configuration
Operating System	Windows
IDE	Visual Studio Code
Language	Python 3.14 (64-bit)
Automation Framework	Selenium WebDriver
Browser	Google Chrome
Test Page	https://the-internet.herokuapp.com/dynamic_loading/1

4. Synchronization Approach

After the Start button is clicked, the page performs a dynamic loading operation. The target text is initially hidden and becomes visible only after the loading process completes. Instead of pausing execution for a fixed amount of time, the script uses an explicit wait.

`WebDriverWait` repeatedly checks the specified expected condition until it becomes true or the timeout is reached. In this test, the condition is `visibility_of_element_located()`, which ensures that the text element is visible before its text is extracted.

5. Test Execution Steps

Step 1: Launch Chrome through Selenium WebDriver.

Step 2: Navigate to the dynamic loading page.

Step 3: Maximize the browser window.

Step 4: Locate and click the Start button.

Step 5: Create a WebDriverWait with a 10-second timeout.

Step 6: Wait until the Hello World element becomes visible.

Step 7: Extract the visible text using the text property.

Step 8: Assert that the extracted text is exactly Hello World!.

Step 9: Print the result and close the browser.

6. Key Synchronization Code

```
wait = WebDriverWait(driver, 10)

text_element = wait.until(
    EC.visibility_of_element_located(
        (By.XPATH, "//div[@id='finish']/h4")
    )
)

result_text = text_element.text

assert result_text == "Hello World!"
```

The script waits for the condition instead of using a fixed delay. Therefore, `time.sleep()` is not required anywhere in the test.

7. Complete Test Script

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Launch Chrome browser
driver = webdriver.Chrome()

# Open dynamic loading page
driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")

# Maximize browser
driver.maximize_window()

# Click the Start button
start_button = driver.find_element(By.XPATH, "//button[text()='Start']")
start_button.click()

# Create explicit wait
wait = WebDriverWait(driver, 10)

# Wait until the Hello World text is visible
text_element = wait.until(
    EC.visibility_of_element_located(
        (By.XPATH, "//div[@id='finish']/h4")
    )
)

# Extract the text
result_text = text_element.text

print("Extracted Text:", result_text)

# Validate the extracted text
assert result_text == "Hello World!"

print("Assignment 2 Passed!")

# Close browser
driver.quit()
```

8. Execution Evidence

The screenshots below document the actual execution of the test: the browser shows the dynamic-loading page and the VS Code terminal shows the extracted text and successful assertion.

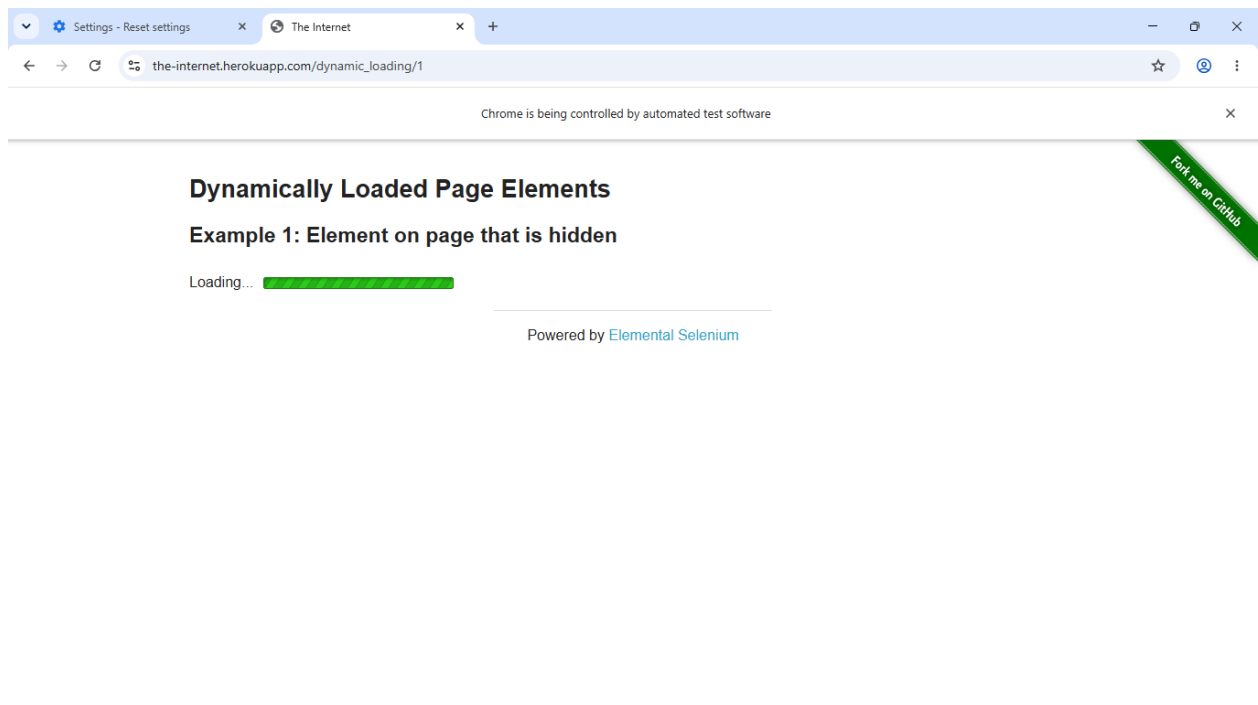


Figure 1 – Dynamic loading page during Selenium execution.

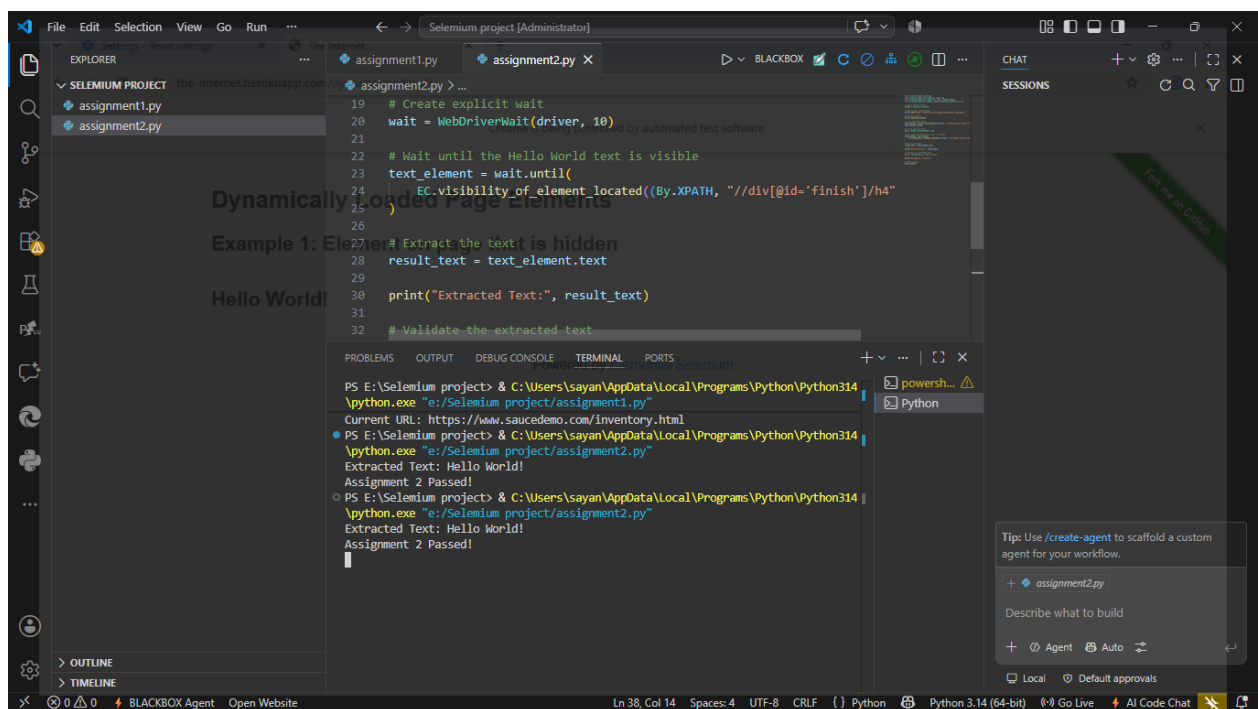


Figure 2 – VS Code terminal showing the extracted text and 'Assignment 2 Passed!' result.

9. Test Result

Validation Point	Expected Result	Observed Result	Status
Dynamic page opened	Page loads successfully	Dynamic loading page opened	PASS
Start action	Loading begins	Start action executed	PASS
Explicit wait	Wait for visible text	WebDriverWait + visibility condition	PASS
Fixed delay restriction	No time.sleep()	No time.sleep() used	PASS
Text extraction	Hello World!	Hello World!	PASS
Assertion	Text matches expected	Assertion passed	PASS
Overall execution	All checks pass	Assignment 2 Passed!	PASS

10. Conclusion

The Synchronization & Explicit Waits assignment was completed successfully using Selenium WebDriver with Python. The test interacted with dynamically loaded content and synchronized execution using WebDriverWait together with visibility_of_element_located(). The dynamic text was extracted only after it became visible, and the resulting value was validated using an assertion.

The assignment requirement prohibiting time.sleep() was followed. The terminal execution confirms that the extracted text was 'Hello World!' and that the test completed with the message 'Assignment 2 Passed!'.

Final Status: PASSED

End of Report